

Grocery Expiration Tracking System

Anirudh Chebolu

Atul Prem Patnala

Sathvik Thota

Tahseen Hussain

Leslie Aguilarpiedra

Table of Content

• Introduction.....	3
○ Overview.....	3
○ Purpose.....	3
○ Target Users.....	3
• Architecture Design Diagram.....	3
○ Presentation Layer (Frontend).....	4
○ Application Layer (Backend).....	4
○ Data Layer (Database).....	5
○ External Services Layer (Firebase + Email API).....	5
• UI Design.....	5
○ UI Flow Diagram.....	6
○ UI Wireframes.....	7
○ Dashboard Page.....	7
○ Add Item Page.....	8
○ Edit Item Page.....	9
• User Experience (UX) Considerations.....	9
○ Intuitive Navigation & Workflow.....	9
○ Clear & Informative Feedback.....	10
○ Efficient Data Entry & Management.....	10
○ Accessibility & Readability.....	10
○ Mobile Responsiveness & Cross-Device Usability.....	10
• Low-Level Design.....	10
○ ER Diagram.....	11
○ Class Diagram.....	12
○ Component Diagram.....	13
○ Hosting & Deployment.....	13
○ Frontend (React.js).....	13
○ Backend (Node.js + Express.js).....	14
○ Database (MySQL).....	14
○ Notification Service (Firebase).....	14
○ External APIs.....	14
○ How Components interact.....	14
• High-Level Architecture.....	15
○ System Context Diagram.....	16
○ Deployment Diagram.....	17
○ Key Components.....	18
○ Flow of Data.....	18
• Conclusion.....	18

Introduction

Overview

The Grocery Expiration Tracking System is designed to help users efficiently manage their household grocery inventory by tracking expiration dates. It provides a streamlined way to monitor food items, reducing waste and ensuring fresh consumption. Users can add grocery items along with their purchase and expiry dates, and the system automatically notifies them when items are about to expire.

Purpose

The primary goal of this system is to offer a simple yet effective solution for tracking grocery expiration dates. By sending timely notifications, it helps users stay organized, avoid food wastage, and plan their meals accordingly. The system integrates essential functionalities such as user authentication, inventory management, and automated reminders, ensuring a smooth user experience.

Target Users

This system is designed for households and individual users who want to maintain an organized grocery inventory. It is also beneficial for small-scale food storage areas, such as shared kitchens or dormitories, where tracking food expiration is crucial. The system is user-friendly, making it suitable for individuals of all ages with minimal technical knowledge.

Architecture Design Diagram:

The Architecture Design Diagram provides an overview of the structural organization of the Grocery Expiration Tracking System, highlighting the key components and their interactions. It follows a multi-layered architecture, consisting of the Presentation Layer (Frontend), Application Layer (Backend), Data Layer (Database), and External Services Layer. Each layer is responsible for a specific set of functionalities, ensuring separation of concerns and modularity. The frontend interacts with the backend via REST API calls, while the backend handles business logic, authentication, and notifications. The database stores essential data such as grocery items, user information, and notifications, while external services manage email notifications. This architecture ensures scalability, maintainability, and efficient data flow between system components.

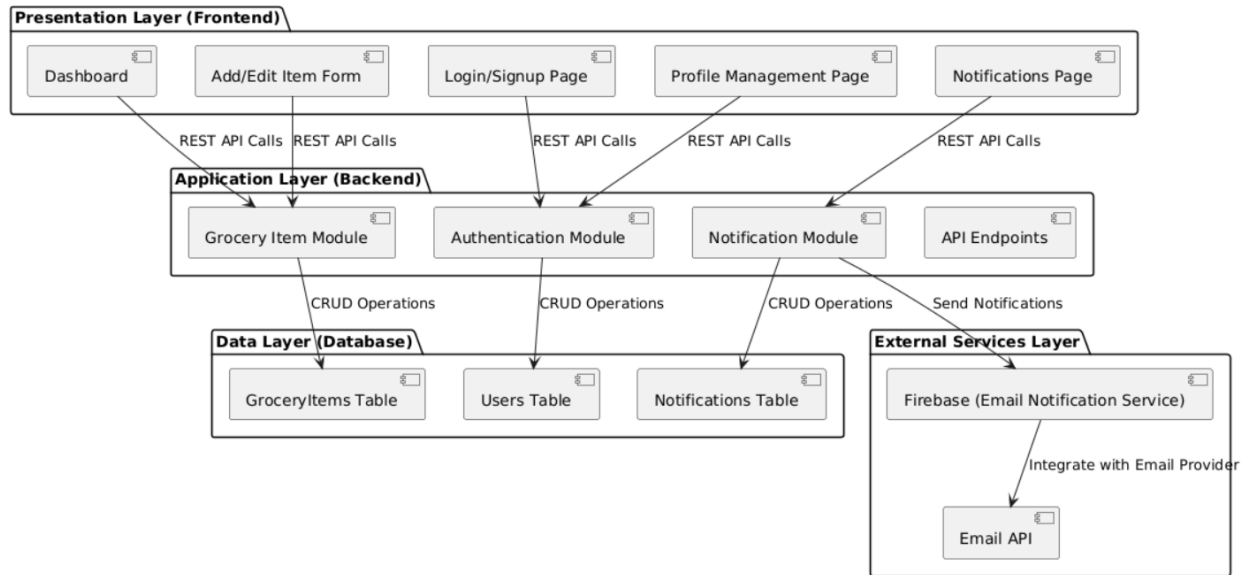


Figure 1: Architecture Diagram of system

The architecture diagram (Figure 1) above represents a multi-layered Grocery Expiration Tracking System, consisting of four key layers.

1. Presentation Layer (Frontend)

- This is the user interface (UI) that interacts with users.
- It includes various screens/pages:
 - Dashboard (shows items and expiry details).
 - Add/Edit Item Form (for adding or modifying grocery items).
 - Login/Signup Page (for user authentication).
 - Profile Management Page (to update user details).
 - Notifications Page (to view upcoming expiration alerts).
- It communicates with the backend through REST API calls.

2. Application Layer (Backend)

- The backend server handles business logic and processes user requests.
- Main backend modules include:
 - Grocery Item Module (handles item-related actions like adding, updating, and retrieving grocery data).
 - Authentication Module (handles user login, signup, and authentication).
 - Notification Module (triggers alerts for expiring items).
 - API Endpoints (provides structured access for frontend requests).
- It performs CRUD operations (Create, Read, Update, Delete) on the database.

3. Data Layer (Database)

- This is where all grocery, user, and notification data is stored.

- Three main database tables:
 - GroceryItems Table (stores grocery item details).
 - Users Table (stores user credentials and profile information).
 - Notifications Table (stores notification records for expired or soon-to-expire items).
- The backend interacts with the database to store and retrieve data.

4. External Services Layer (Firebase + Email API)

- Firebase is used as an email notification service.
- It sends expiration alerts via email when an item is about to expire.
- Firebase integrates with an Email API (such as SendGrid or Mailgun) to deliver emails.

UI Design

The UI Workflow Diagram (Figure 2) visually represents the navigation and interaction flow of the Grocery Expiration Tracking System, outlining how users move between different screens. It provides a structured overview of key user actions, such as logging in, adding grocery items, tracking expiry dates, and managing notifications. This diagram ensures a seamless user experience by defining intuitive pathways, minimizing confusion, and optimizing accessibility. By mapping out the UI structure, it helps in designing a user-friendly interface that aligns with system functionalities, improving usability and efficiency.

UI Flow Diagram:

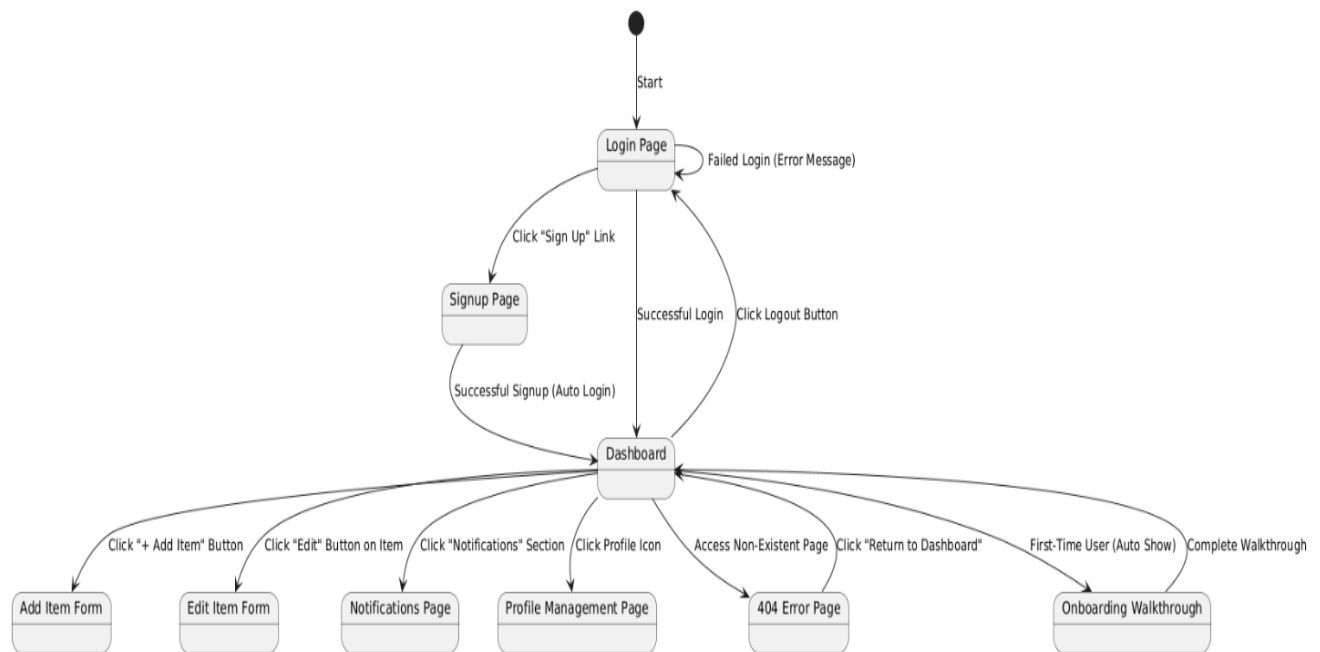


Figure 2: UI Workflow Diagram

The UI Workflow Diagram (Figure 2) visually represents the navigation and interaction flow of the Grocery Expiration Tracking System.

Login Page → Dashboard

- After successful login, the user is redirected to the Dashboard.
- If login fails, an error message appears.

Signup Page → Dashboard

After signing up, the user is automatically logged in and redirected to the Dashboard.

Dashboard → Add Item

- Clicking the “+” Add Item button opens the Add Item Form.

Dashboard → Edit Item

- Clicking the Edit button on any grocery item opens the Edit Item Form.

Dashboard → Notifications Page

- Clicking on the "Notifications" section navigates to the Notifications Page.

Dashboard → Profile Management

- Clicking on the Profile Icon navigates to the Profile Management Page.

Dashboard → Logout

- Clicking the Logout button logs the user out and redirects to the Login Page.

Error Page Handling

- If the user accesses a non-existent page, they land on the 404 Error Page with an option to return to the Dashboard.

First-Time User Onboarding

- New users are shown an Onboarding Walkthrough Overlay on their first login, with step-by-step guidance on adding items and tracking expiry.

UI Wireframes:

Dashboard Page

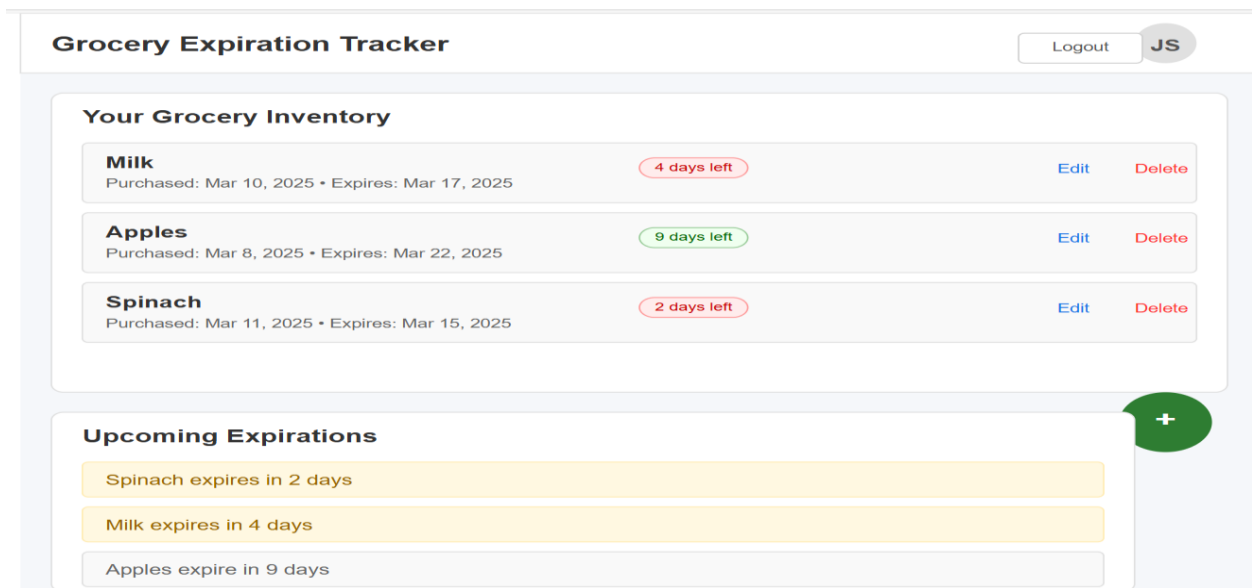


Figure 3: Wireframe of Dashboard Page

This page (Figure 3) is the main landing page after login. It displays a list of grocery items with details such as name, purchase date, and expiry date. Users can see upcoming expirations and take quick actions like editing or deleting an item. A floating “Add Item” button allows users to enter new grocery items seamlessly.

Add Item Page:

Add New Item

Item Name:

Purchase Date:

Expiration Date:

Cancel Save

Figure 4: Wireframe of Add new Item page

This page (Figure 4) provides a simple form where users can enter a new grocery item by specifying the name, purchase date, and expiry date. The page ensures a clean layout with validation to prevent errors, such as selecting an expiry date earlier than the purchase date.

Edit Item Page:

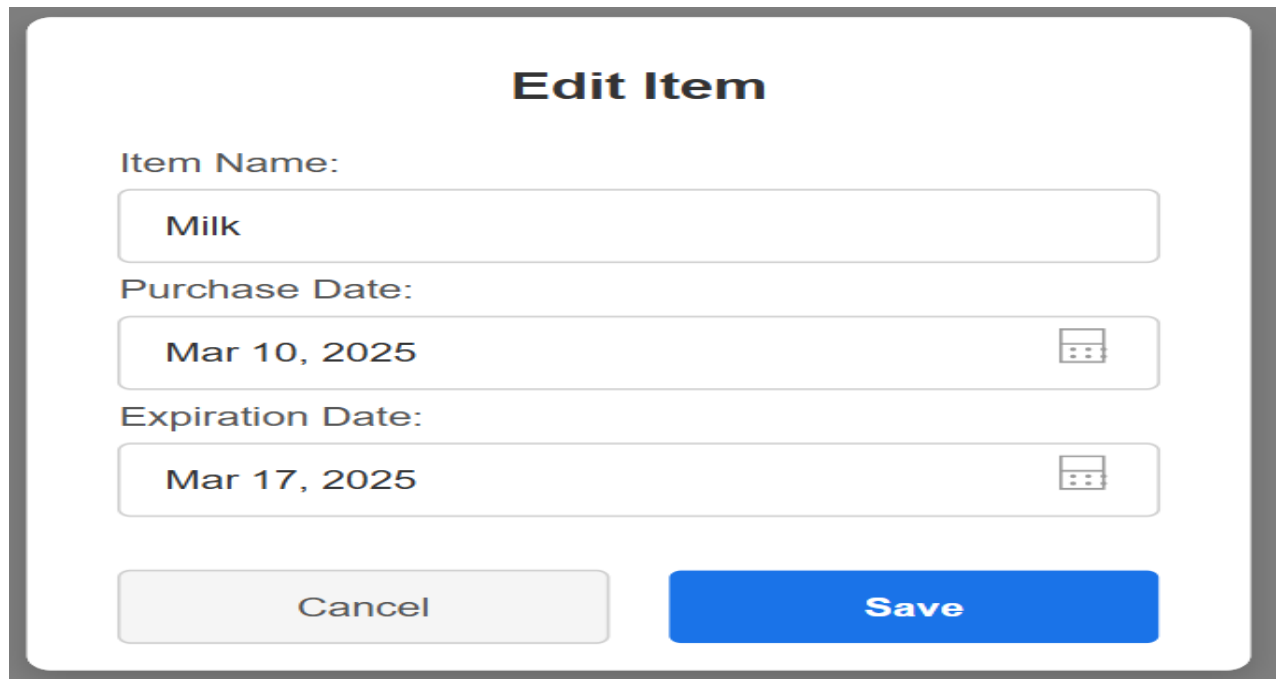
A wireframe of the 'Edit Item' page. The title 'Edit Item' is centered at the top. Below it, there are three input fields: 'Item Name:' with the text 'Milk', 'Purchase Date:' with the text 'Mar 10, 2025', and 'Expiration Date:' with the text 'Mar 17, 2025'. Each date field has a small calendar icon on the right. At the bottom, there are two buttons: a light gray 'Cancel' button and a blue 'Save' button.

Figure 5: Wireframe of Edit Item page

This page (Figure 5) allows users to modify existing grocery items. It pre-fills the form with the current details of the selected item, enabling users to make updates without re-entering all information. The interface remains consistent with the Add Item page for familiarity.

User Experience (UX) Considerations for Grocery Expiration Tracking System

The Grocery Expiration Tracking System is designed to provide a simple, efficient, and user-friendly experience. Below are key UX considerations to ensure smooth interaction and usability:

1. Intuitive Navigation & Workflow

- A straightforward layout allows users to quickly add, track, and manage grocery items without unnecessary complexity.
- Navigation menus and buttons are placed in accessible locations to minimize user confusion.
- The system follows a logical flow: users log in → add grocery items → track expiration dates → receive notifications.

2. Clear and Informative Feedback

- Users receive real-time alerts for expiring items (e.g., "Milk expires in 2 days!") through notifications, ensuring proactive action.

- Error messages prevent mistakes, such as entering an expiration date earlier than the purchase date.
- Confirmation dialogs appear before deleting an item, preventing accidental data loss.

3. Efficient Data Entry & Management

- Pre-filled date fields (e.g., defaulting purchase date to the current day) speed up item entry.
- Dropdowns or autocomplete suggestions help users enter grocery names faster.
- Edit and delete functions allow users to update or remove expired items easily.

4. Accessibility & Readability

- A clean and minimal design ensures focus on key information without distractions.
- Font sizes and spacing are optimized for readability, ensuring ease of use for all users.
- Consistent UI elements (buttons, input fields) make interaction predictable and user-friendly.

5. Mobile Responsiveness & Cross-Device Usability

- The system is designed to be mobile-friendly, allowing users to track their groceries from any device.
- Touch-friendly buttons and forms improve usability on smaller screens.

Low-Level Design

The Low-Level Design (LLD) of the Grocery Expiration Tracking System provides a detailed view of the system's internal structure, focusing on database schema, class relationships, and component interactions. This section includes an Entity-Relationship (ER) Diagram, Class Diagram, and Component Diagram, each offering a specific perspective on the system's design. The ER Diagram defines how grocery items, users, and notifications are structured within the database. The Class Diagram outlines object-oriented relationships, showing how different system entities interact. The Component Diagram illustrates the system's modular structure, detailing how various components, such as inventory management, authentication, and notifications, communicate with each other. These diagrams ensure a clear, maintainable, and scalable design for efficient system implementation.

ER Diagram:

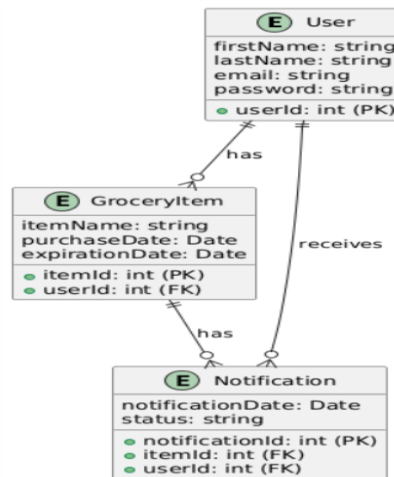


Figure 6: ER Diagram

The ER Diagram in (Figure 6) represents the Grocery Expiry Tracker system, consisting of three main entities:

User: Stores user details (userId, firstName, lastName, email, password). A user can add multiple grocery items and receive notifications.

Grocery Item: Tracks grocery items (itemId, itemName, purchaseDate, expirationDate, userId). Each item belongs to one user and can generate multiple notifications.

Notification: Stores expiry alerts (notificationId, notificationDate, status, itemId, userId). Each notification is linked to a specific grocery item and user.

Relationships:

- User - GroceryItem (1:M): A user can have multiple grocery items.
- GroceryItem - Notification (1:M): Each grocery item can trigger multiple notifications.
- User - Notification (1:M): A user can receive multiple notifications.

Class Diagram:

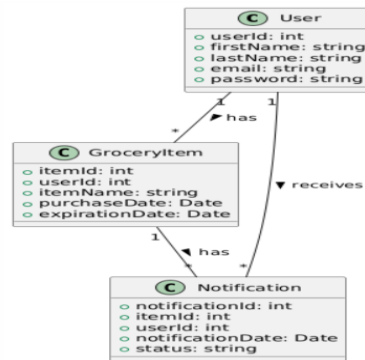


Figure 7: Class Diagram

The Class Diagram in (Figure 7) represents the Grocery Expiry Tracker system, outlining three main classes and their relationships:

User:

- Attributes: userId, firstName, lastName, email, password
- A user can add multiple grocery items and receives notifications about expiry.

Grocery Item:

- Attributes: itemId, userId, itemName, purchaseDate, expirationDate
- Each grocery item belongs to a user and can trigger multiple notifications.

Notification:

- Attributes: notificationId, itemId, userId, notificationDate, status
- Each notification is linked to a specific grocery item and user.

Relationships:

- User - GroceryItem (1:M): A user can have multiple grocery items.
- GroceryItem - Notification (1:M): Each grocery item can have multiple notifications.
- User - Notification (1:M): A user receives multiple notifications about their groceries.

Component Diagram:

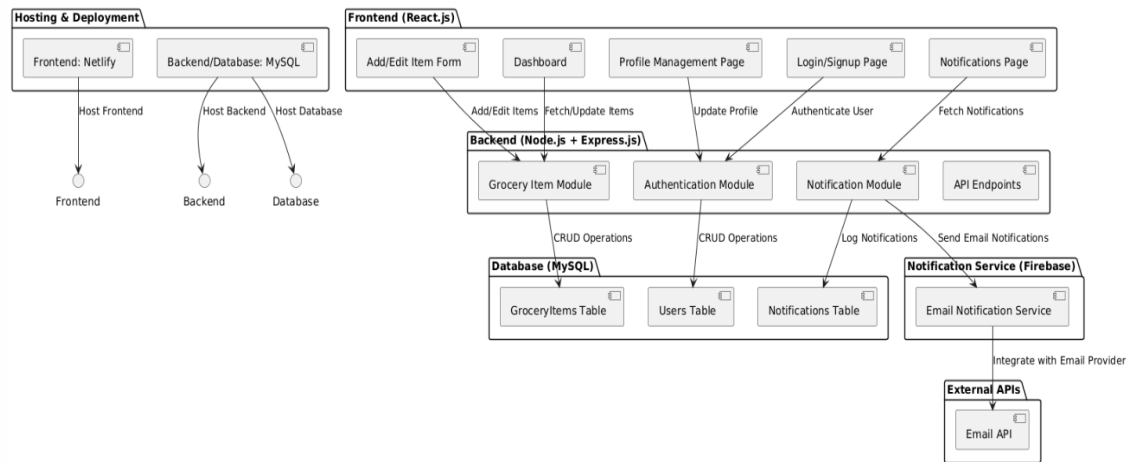


Figure 8: Component Diagram

This (Figure 8) Component Diagram represents the architecture of the Grocery Expiry Tracker system, showing how different components interact with each other. Below is a breakdown of the key elements and their relationships:

1. Hosting & Deployment

- The system is hosted using:
 - Frontend on Netlify – The React.js-based frontend is deployed here.
 - Backend & Database on MySQL – The backend and database are hosted separately.
- These components:
 - Host Frontend connects to the frontend.
 - Host Backend connects to the backend services.
 - Host Database ensures data storage is managed efficiently.

2. Frontend (React.js)

- This is the user-facing part of the system.
- Key pages include:
 - Add/Edit Item Form – Allows users to add or update grocery items.
 - Dashboard – Displays stored grocery items and expiry information.
 - Profile Management Page – Enables users to manage their profile.
 - Login/Signup Page – Handles authentication.
 - Notifications Page – Displays notifications about items nearing expiry.
- These pages interact with the backend for various operations like:
 - Adding/updating items.
 - Authenticating users.
 - Fetching notifications.

3. Backend (Node.js + Express.js)

- Acts as the intermediary between the frontend and the database.
- Contains different modules:
 - Grocery Item Module – Handles CRUD (Create, Read, Update, Delete) operations for grocery items.
 - Authentication Module – Manages user authentication (login/signup).
 - Notification Module – Logs and fetches expiry notifications.
 - API Endpoints – Provides interfaces for frontend interaction.

4. Database (MySQL)

- Stores all data required for the system.
- Composed of:
 - GroceryItems Table – Stores item details (name, purchase date, expiry date).
 - Users Table – Stores user credentials and profile information.
 - Notifications Table – Logs expiry notifications.

5. Notification Service (Firebase)

- Manages email notifications about expiring grocery items.
- Email Notification Service:
 - Logs expiry notifications from the backend.
 - Sends email notifications via an External API.

6. External APIs

- Email API:
 - Used by the Notification Service to send email alerts to users when an item is about to expire.

How Components Interact

1. User Interactions (Frontend):
 - a. The user accesses the system via the frontend (React.js).
 - b. They log in, add/edit items, check expiry notifications, and manage their profile.
2. Backend Processing:
 - a. User actions trigger API requests to the Node.js & Express.js backend.
 - b. The backend processes the request, updates the database, and fetches relevant data.
3. Database Operations:
 - a. The backend interacts with the MySQL Database to store or retrieve grocery items, user details, and notifications.
4. Notification Handling:

- a. The backend logs expiry notifications.
- b. Notifications are sent to Firebase Notification Service, which then integrates with an external Email API to notify users.

High-Level Architecture

The High-Level Architecture of the Grocery Expiration Tracking System defines the key system components, their interactions, and the overall deployment structure. This section includes a System Context Diagram and a Deployment Diagram to illustrate how different modules communicate and operate within the system. The architecture ensures modularity, scalability, and efficient data handling by clearly separating the frontend (user interface), backend (business logic), database (data storage), and external services (notifications and authentication). These diagrams provide a visual representation of how the system components interact, ensuring smooth functionality and maintainability.

System Context Diagram:

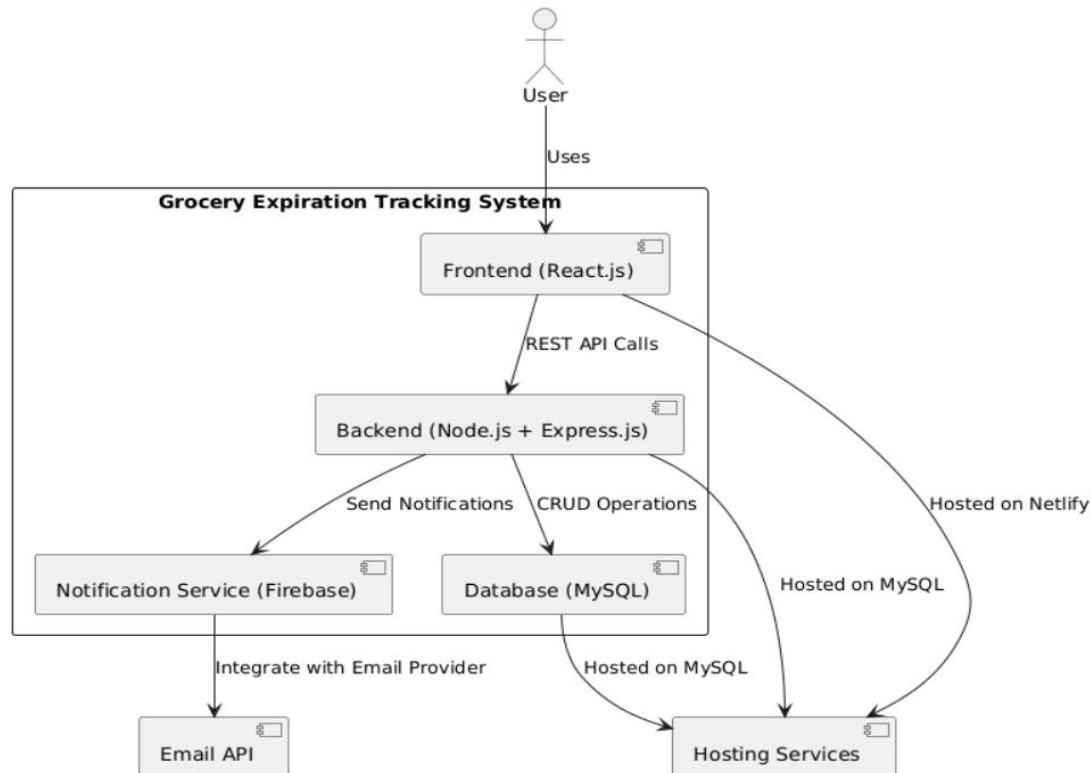


Figure 9: System Context Diagram

This (Figure 9) System Architecture Diagram represents how different components in the Grocery Expiration Tracking System interact.

Key Components & Flow:

1. User Interaction:
 - a. The user accesses the system via a frontend web app (React.js).
 - b. The frontend is hosted on Netlify and makes REST API calls to the backend.
2. Backend Processing:
 - a. The backend (Node.js + Express.js) handles requests and interacts with the database and notification system.
 - b. It performs CRUD (Create, Read, Update, Delete) operations on the MySQL database.
3. Database (MySQL):
 - a. Stores grocery items, user information, and expiration dates.
 - b. Hosted on MySQL (cloud or local server).
4. Notification Service (Firebase):
 - a. The backend sends notifications about expiring items to Firebase.
 - b. Firebase integrates with an email API to send reminders to users.
5. Hosting Services:
 - a. The frontend is hosted on Netlify.
 - b. The backend and database are hosted on MySQL.

How it Works:

1. The user logs in and adds grocery items (name, purchase date, expiry date).
2. The frontend sends this data to the backend via REST APIs.
3. The backend updates the database and checks for upcoming expirations.
4. If an item is near expiry, the backend triggers a notification via Firebase.
5. The notification is sent to the user via email using an external email API.

Deployment diagram:

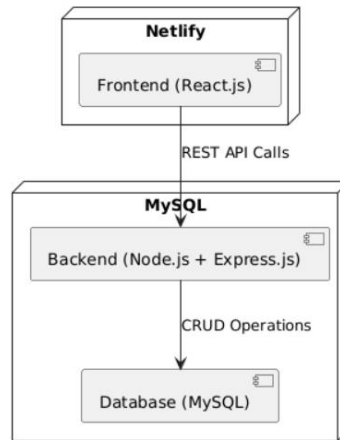


Figure 10: Deployment Diagram

This (Figure 10) is the Deployment Diagram that illustrates a web application hosted on Netlify, where a React.js frontend communicates with a Node.js + Express.js backend via REST API calls. The backend interacts with a MySQL database to perform CRUD operations for data management.

Key Components:

1. Frontend (React.js) on Netlify:
 - a. The user interacts with the system through a React.js frontend, which is hosted on Netlify.
 - b. The frontend makes REST API calls to communicate with the backend.
2. Backend (Node.js + Express.js) on MySQL Server:
 - a. The backend is built with Node.js and Express.js and is responsible for handling API requests.
 - b. It processes user actions, such as adding, updating, and retrieving grocery data.
3. Database (MySQL):
 - a. The backend interacts with a MySQL database, where grocery item details (e.g., name, purchase date, expiry date) are stored.
 - b. The backend performs CRUD (Create, Read, Update, Delete) operations on this database.

Flow of Data:

1. User interacts with the frontend (e.g., adds an item).
2. Frontend sends API requests to the backend via REST API calls.
3. Backend processes the request and performs necessary database operations.
4. Database stores and retrieves the required data.
5. Backend sends the response back to the frontend.

Conclusion

This Project Detailed Design Document provides a complete overview of the Grocery Expiry Tracker's architecture, high and low-level designs, and user interface specifications. It is designed to meet all the criteria outlined project requirements, ensuring a solid foundation for project development and successful delivery.